

Exercise 4: Pattern Matching 30 Points Possible

11/14/2025



Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

10/29/2025 to 11/17/2025

▼ Details

In this assignment, you will implement some code related to a Turtle program using pattern matching, destructuring, and pipelining to experiment with ideas from functional programming.

Setup/Submission

- Setup: [Turtle.exs \(https://ufl.instructure.com/courses/540435/files/101637358?wrap=1\)](https://ufl.instructure.com/courses/540435/files/101637358?wrap=1)  [\(https://ufl.instructure.com/courses/540435/files/101637358/download?download_frd=1\)](https://ufl.instructure.com/courses/540435/files/101637358/download?download_frd=1) contains the provided code. You can use an online editor such as [Tutorialspoint !\[\]\(3ae06528cbf191565604ae076c36537e_img.jpg\) \(https://www.tutorialspoint.com/compilers/online-elixir-compiler.htm\)](https://www.tutorialspoint.com/compilers/online-elixir-compiler.htm) (save regularly!), or setup a local environment for Elixir (possible but not as straightforward as other languages).
- Submission: You will submit your updated `Turtle.exs` file. You should only implement the given functions within `Turtle`; the surrounding code structure and logic should be left unchanged (but you may add additional tests as desired).

Elixir Overview

Elixir is a functional programming language with syntax based on Ruby and semantics based on Erlang. Elixir is known for its use of the Actor model and capabilities for distributed systems, and has been used by companies like [Discord](https://elixir-lang.org/blog/2020/10/08/real-time-communication-at-scale-with-elixir-at-discord/)  [\(https://elixir-lang.org/blog/2020/10/08/real-time-communication-at-scale-with-elixir-at-discord/\)](https://elixir-lang.org/blog/2020/10/08/real-time-communication-at-scale-with-elixir-at-discord/).

- The [Elixir](https://hexdocs.pm/elixir/introduction.html)  [docs](https://hexdocs.pm/elixir/introduction.html) have good coverage over basic syntax/structure. The main syntax pieces to be familiar with are:
 - Atoms can be thought of as the runtime value of an identifier, like `:atom`. They're used similarly to string constants in other languages.
 - Tuples work similarly to other languages but use braces, like `{:ok, position}`.
 - Lists use the standard square bracket syntax, like `[1, 2, 3]`, but use a cons list structure that can be destructured like `[head | tail]` (equivalent to `[head, *tail]` in Python).
- Elixir, like many functional languages, allows pattern matching with the function declaration itself to overload functions based on argument structure. Overloads use the first matching declaration, i.e. *functions declared earlier have higher priority*. See docs for [Pattern Matching](https://hexdocs.pm/elixir/pattern-matching.html)  [and](https://hexdocs.pm/elixir/pattern-matching.html) [Function Definitions](https://hexdocs.pm/elixir/modules-and-functions.html#function-definition)  [for details](https://hexdocs.pm/elixir/modules-and-functions.html#function-definition).
- Elixir also supports a feature called Pipelining, which is syntax sugar that allows writing a nested function like `outer(inner(expr))` as `expr |> inner() |> outer()` - the left value is "piped" as the first argument to the next function. Visually, this results in syntax similar to methods but allows any function to be used (though behavior is not identical to methods - why?).

Part 1: Pattern Matching

Implement the remaining definitions for the `move` function to support the instructions below. Your solution **must** leverage pattern matching and destructuring

for *all* conditional logic - i.e., do not use `if` and instead use a separate function definition to handle conditional logic via pattern matching.

Instruction	Description
<code>forward <steps></code>	Moves the turtle forward by a certain number of steps based on the turtle's current direction (i.e., when facing north the turtle move x steps north). The number of steps cannot be zero (but can be negative) and in this case should return the error result with the message <code>"invalid forward steps (0)"</code> .
<code>right</code>	Turns the turtle to the right, changing it's direction.
<code>left</code>	Turns the turtle to the left, changing it's direction.
<code>jump <x> <y></code>	The turtle (somehow) jumps to the position <code>{x, y}</code> without changing direction. For safety, jumping into the <code>{7, 7}</code> wormhole (see below) is invalid and returns an error result with the message <code>"invalid jump into wormhole"</code> .
<code>_</code> (invalid)	Returns an error result with the message <code>"invalid instruction <instruction>"</code> . The correct formatting for the instruction has already been implemented (via <code>str(instruction)</code> / <code>inspect instruction</code>).
wormhole (<code>{7, 7}</code>)	If the turtle ever <u>concludes</u> an instruction at the position <code>{7, 7}</code> (moving through it is fine), the turtle enters a wormhole. The turtle exits the wormhole at <code>{-7, -7}</code> facing the opposite direction <code>{-dx, -dy}</code>. Hint: It seems like we want to prioritize checking the turtle's position over processing instructions...

Part 2: Pipelining

Implement the definition for the `parse` function, which parses an input program contained labeled instructions and transforms it into a map of function definitions of names to instructions (see the example below). The parsing of individual lines has been implemented for you; your responsible is the overarching transformation.

You solution **must** leverage pipelining, not variables, and should ultimately be a single expression containing multiple pipelines. The [Enum](https://hexdocs.pm/elixir/1.12/Enum.html#reduce/3)  (<https://hexdocs.pm/elixir/1.12/Enum.html#reduce/3>) module contains definitions for common map/filter/reduce operations. String, List, Tuple, and Map may be helpful as well.

- Each line is of the form `<name>: <instruction> [arguments...]`. This has been implemented for you; you do not need to worry about invalid input here. Lines are separated by newlines and/or carriage returns.
- Programs may also contain comments (starting with `#`), which simply ignores any input until the next line.
- Whitespace from newlines, empty lines, and indentation should be ignored.

```
#zoom zoom!  
  
uturn: right  
uturn: right  
  
zoom: forward 1000  
zoom: uturn  
zoom: forward 2000  
zoom: uturn  
zoom: forward 3000  
  
main: zoom
```

Choose a submission type



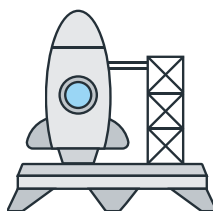
Upload



Google Drive



More



Choose a file to upload
File permitted: EXS



or

 Canvas Files



(<https://ufl.instructure.com/courses/540435/modules/items/12223228>)

Assignment
(<https://ufl.ins>)